AIR QUALITY PREDICTION AND HEALTH ADVISORY SYSTEM

A PROJECT REPORT

for

U23AM491 – MACHINE LEARNING

Submitted by

SURIYA A **722823243144**

in partial fulfillment for the award of the

degree of

BACHELOR OF TECHNOLOGY

in

ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

SRI ESHWAR COLLEGE OF ENGINEERING

(An Autonomous Institution, Affiliated to Anna University - Chennai - 600 025)

NOVEMBER / DECEMBER 2025

DECLARATION

I declare that the work entitled **"AIR QUALITY PREDICTION AND HEALTH ADVISORY SYSTEM"** is submitted in partial fulfillment of the requirement for the award of the degree in B. Tech. (AI&DS)., Anna University Chennai, is a record of the my own work carried out by me during the academic year 2025-2026 under the supervision and guidance of **Dr. G. Sathish Kumar,** Associate Professor, Department of Artificial Intelligence and Data Science, Sri Eshwar College of Engineering, Coimbatore. The extent and source of information are derived from the existing literature and have been indicated through the dissertation at the appropriate places. The matter embodied in this work is original and has not been submitted for the award of any other degree or diploma, either in this or any other University.

Student Name	**Register Number**	**Signature**
SURIYA A	722823243144	

I certify that the declaration made above by the candidate is true.

Dr. G. Sathish Kumar
Associate Professor
Department of AI&DS

BONAFIDE CERTIFICATE

Certified that this project report **"AIR QUALITY PREDICTION AND HEALTH ADVISORY SYSTEM"** is the bonafide work of **"SURIYA A (722823243144)"** who carried out the project work under my supervision.

SIGNATURE

Dr. M. MOHAMMED MUSTAFA
Head of the Department
Associate Professor & Head
Dept. of Artificial Intelligence and Data Science
Sri Eshwar College of Engineering, Coimbatore

SIGNATURE

Dr. G. SATHISH KUMAR
Supervisor
Associate Professor
Dept. of Artificial Intelligence and Data Science
Sri Eshwar College of Engineering, Coimbatore

Submitted for the Project viva-voce examination held on _____

INTERNAL EXAMINER **EXTERNAL EXAMINER**

ACKNOWLEDGEMENT

I thank the Almighty for giving me an opportunity to study in a prestigious institution and grace to complete this project successfully.

I also express my gratitude to our Chairperson **Mr. R. Mohanram** and Director **Mr. R. Rajaram,** Sri Eshwar College of Engineering for their endeavor to provide all the facilities we require and the interest they showed in the welfare of the students.

I wish to express my sincere thanks to **Dr. Sudha Mohanram,** the **Principal & Joint Managing Trustee** of our college for providing the necessary infrastructure facilities.

I express my sincere gratitude to **Dr. M. Mohammed Mustafa** the **Head of the Department,** Artificial Intelligence and Data Science**,** for his guidance and providing valuable suggestion and all necessary facilities at the right time for doing this project.

I express my gratitude and sincere thanks to my guide **Dr. G. Sathish Kumar, Associate Professor,** Dept. of Artificial Intelligence and Data Science, for his valuable suggestions and constant encouragement for successful completion of this project.

I would also extend my hearty thanks to my family members and all the friends who encouraged me to do this project successfully.

TABLE OF CONTENTS

ABSTRACT

Air pollution has become one of the most pressing environmental and public health challenges in the modern world. Rising industrialization, urban traffic emissions, and population growth have significantly deteriorated air quality, leading to severe respiratory, cardiovascular, and environmental impacts. To address this concern, this project presents an end-to-end Air Quality Monitoring and Prediction System that combines IoT-based sensor simulation and Machine Learning (ML) to classify and analyze air quality levels in real time.

Several machine learning algorithms are evaluated, including Random Forest and XGBoost, to identify the most accurate and stable model for classification. The final model categorizes the air quality into five distinct levels — *Good, Moderate, Unhealthy for Sensitive Groups, Unhealthy,* and *Very Unhealthy* — following standard AQI (Air Quality Index) guidelines.

Once trained, the best-performing model is serialized and deployed through an interactive Streamlit dashboard. The dashboard provides a user-friendly interface where users can:

- Manually input sensor values to predict the air quality in real time.
- Perform Exploratory Data Analysis (EDA) to visualize trends, correlations, and pollution distributions.
- Receive personalized health advisories based on the predicted AQI category, helping citizens make informed decisions regarding outdoor activities.

This integrated system serves not only as a predictive and advisory tool for individuals but also as a prototype for smart city applications, enabling authorities to track pollution patterns and take preventive actions.

LIST OF FIGURES

LIST OF ABBREVIATIONS

AQI	Air Quality Index
ML	Machine Learning
EDA	Exploratory Data Analysis
CSV	Comma Separated Values
RF	Random Forest
XGB	XGBoost
ppm	parts per million

CHAPTER 1

INTRODUCTION

Air pollution has emerged as one of the most severe environmental and public health challenges of the 21st century. Rapid urbanization, industrial growth, and vehicular emissions have significantly deteriorated air quality, especially in densely populated areas. Pollutants such as carbon monoxide (CO), nitrogen oxides (NOx), and volatile organic compounds (VOCs) contribute to respiratory and cardiovascular diseases, leading to millions of deaths each year.

This project presents an Air Quality Detection and Health Advisory System that combines IoT-based sensor simulation with Machine Learning (ML) analytics. Using Tinkercad, an Arduino Uno and MQ gas sensor are simulated to collect environmental data. These readings, along with historical air quality datasets, train ML models like Random Forest and XGBoost to predict Air Quality Index (AQI) levels ranging from *Good* to *Very Unhealthy*.

The trained model is deployed on a Streamlit dashboard, offering visualization, prediction, and Exploratory Data Analysis (EDA) features. Users can input sensor data or upload files for batch prediction and receive health advisories based on AQI outcomes. This scalable and educational prototype demonstrates the potential for real-world use in smart cities, environmental monitoring, and public health awareness.

Furthermore, the integration of real-time simulation with machine learning ensures a cost-effective and practical approach for understanding pollution trends. By bridging the gap between sensor data and actionable insights, the system promotes environmental consciousness among users and supports data-driven decision-making for cleaner and healthier communities.

CHAPTER 2

LITERATURE REVIEW

Low-cost gas sensors, particularly those in the **MQ series**, are commonly used in **IoT-based air quality monitoring systems** due to their affordability and ease of integration. However, these sensors require regular **calibration** to maintain accuracy, as their readings are highly influenced by **temperature and humidity variations** (Kumar et al., 2022). Despite these limitations, their widespread adoption has made them ideal for community-level or educational air quality projects.

Machine Learning (ML) algorithms such as **Random Forest, Decision Trees, and Gradient Boosting (XGBoost)** have shown strong performance in handling **tabular environmental datasets** for both classification and regression tasks (Li & Zhao, 2021). These models effectively capture nonlinear relationships among pollutants and meteorological variables, making them suitable for predicting **Air Quality Index (AQI)** categories and pollutant concentrations.

The use of **Streamlit dashboards** facilitates rapid and user-friendly deployment of trained ML models. Streamlit allows developers to build **interactive web applications** that enable users to visualize data trends, make predictions, and analyze results without needing programming expertise (Patel & Singh, 2023). This significantly enhances the accessibility of air quality monitoring tools for researchers, policymakers, and the general public.

Despite these advancements, several **research gaps** remain. Sensor drift over time leads to reduced reliability, and the **lack of large, well-labeled datasets** limits model generalization. Furthermore, **domain adaptation**—adjusting models for **location-specific calibration**—is essential for improving prediction accuracy across diverse environments (Chen et al., 2024). Addressing these challenges is crucial for developing robust, scalable, and real-world air quality prediction systems.

CHAPTER 3

PROBLEM DEFINITION

3.1 EXISTING SYSTEM

Existing air quality monitoring systems mostly provide raw sensor readings in parts per million (PPM) or give simple threshold-based alerts. These readings are often difficult for ordinary users to understand in terms of health safety. Most of the systems lack predictive modeling or machine learning-based classification for air quality levels. They do not map pollutant concentrations to standard Air Quality Index (AQI) categories. Advanced features like data visualization, trend analysis, and Exploratory Data Analysis (EDA) are not available. Batch prediction and automated data processing are also missing. Many setups rely on single-sensor inputs, which reduce accuracy and reliability. Environmental factors such as humidity and temperature often affect sensor readings. No real-time health advisory or alert system is provided to guide users. Overall, existing systems are basic, less informative, and unsuitable for real-time decision-making.

3.2 DISADVANTAGES OF EXISTING SYSTEM

- Difficult Interpretation: Raw sensor readings are shown in PPM values, which are hard for normal users to understand. They do not indicate whether the air is safe or hazardous, making it difficult to take preventive action.
- No Classification Model: Existing systems lack intelligent models that can classify air quality into standard AQI levels such as Good, Moderate, or Unhealthy. Without this, users cannot easily relate readings to health effects.
- Lack of Data Analysis Features: Most systems do not include batch prediction, data visualization, or Exploratory Data Analysis (EDA), which are essential for understanding pollution patterns and system performance.
- Limited Sensor Accuracy: Many low-cost setups depend on a single gas sensor, which reduces reliability. Such sensors are easily influenced by temperature, humidity, and environmental changes, affecting accuracy.

3.3 PROPOSED SYSTEM

The proposed system uses historical air quality datasets to train a machine learning model capable of classifying Air Quality Index (AQI) levels and generating relevant health advisories. The data undergoes preprocessing, including handling missing values and removing outliers. Machine learning algorithms like Random Forest and XGBoost are applied to classify air quality into five categories — Good, Moderate, Unhealthy for Sensitive Groups, Unhealthy, and Very Unhealthy.

A Streamlit dashboard is developed to make the system interactive and accessible. It allows manual input of sensor values and batch predictions using CSV uploads. The dashboard includes an Exploratory Data Analysis (EDA) section with visualizations like histograms and correlation plots to help users understand pollution behavior. It also provides model information, performance metrics, and feature importance for better transparency.A Tinkercad-based Arduino simulation demonstrates the local sensing and alert mechanism. Using an Arduino Uno and MQ gas sensor, the circuit simulates real-time air quality detection and displays readings on an LCD. LEDs and buzzers provide instant alerts when pollution levels exceed safe limits.

3.4 ADVANTAGES OF PROPOSED SYSTEM

- Accurate Prediction: ML models classify air quality and predict pollution levels effectively.
- Health Guidance: Provides clear health advisories based on AQI categories.
- Easy to Use: Streamlit dashboard supports manual input, CSV upload, and visualization.
- Low-Cost Setup: Uses affordable IoT components like Arduino and MQ sensors.
- Scalable Design: Can be extended for smart city and educational applications.

CHAPTER 4

SYSTEM STUDY

4.1 FEASIBILITY STUDY

A feasibility study is conducted to evaluate the practicality, cost-effectiveness, and social acceptance of the proposed system. It ensures that the project is both technically and economically viable before implementation. The feasibility of this project is analyzed under three main categories: economic, technical, and social feasibility.

4.1.1 Economical Feasibility

The system demonstrates strong economic feasibility by utilizing Tinkercad simulation, which eliminates the need for costly hardware during the development phase. It relies on open-source tools such as Python, scikit-learn, XGBoost, and Streamlit, thereby minimizing software expenses. The model can be trained and executed on a standard desktop or laptop without requiring high-end computational resources, reducing infrastructure costs.

4.1.2 Technical Feasibility

The system is technically feasible as it utilizes well-established and open-source tools such as Python, pandas, scikit-learn, XGBoost, Streamlit, and joblib for data analysis, model training, and deployment. Tinkercad and Arduino IDE are employed for simulation and demonstration, providing a realistic yet virtual hardware environment for testing. The model can be efficiently trained on a standard desktop or laptop, as the dataset size and computational requirements are minimal.

4.1.3 Social Feasibility

The system demonstrates strong social feasibility as it enhances public awareness through a user-friendly Streamlit dashboard that visually presents air quality levels and health advisories. By translating complex sensor data into clear, understandable messages, it supports informed decision-making for individuals and communities. The health advisory component aligns with recognized public safety guidelines, ensuring relevance and trust.

CHAPTER 5

SYSTEM REQUIREMENTS SPECIFICATION

5.1 HARDWARE REQUIREMENTS

The hardware setup for this project is designed to be simple, low-cost, and easily replicable for educational and prototype purposes. Instead of using physical components, the system is demonstrated using Tinkercad, an online simulation tool that allows testing and visualization without any real hardware. The components listed below are used to simulate the air quality sensing and alert mechanism effectively:

- Arduino UNO (simulated)
- MQ-series gas sensor (MQ-135 / MQ-2 simulated)
- 16×2 LCD (LiquidCrystal) and potentiometer for contrast
- Jumper wires, breadboard (simulated)

5.2 SOFTWARE REQUIREMENTS

- Arduino IDE
- Tinkercad (Simulation)
- Python 3.11
- Libraries: pandas, sklearn, joblib, matplotlib, streamlit

5.3 SOFTWARE DESCRIPTION

- Arduino IDE – Used to program and simulate the microcontroller setup in Tinkercad. It helps read sensor data, control the LCD, and alerts based on air quality levels.
- Streamlit – Provides an interactive web-based dashboard for visualizing data, performing manual or batch predictions, and displaying health advisories in a user-friendly format.
- Scikit-learn – Used for training and testing machine learning models, enabling accurate classification of air quality into different AQI categories based on pollutant data.

CHAPTER 6

SYSTEM DESIGN

The System Design phase defines how the proposed system will be structured and how its components will interact to achieve the desired functionality.

6.1 SYSTEM ARCHITECTURE

The proposed Air Quality Detection and Health Advisory System consists of two main modules: the Hardware Simulation Module and the Software Module.

In the Hardware Simulation Module, components such as the Arduino UNO, MQ-135 gas sensor, LCD display and potentiometer are connected virtually using Tinkercad. This setup helps simulate real-time air-quality readings and display sensor values on the LCD without the need for physical hardware.

The Software Module involves using Python and Streamlit for data visualization and analysis. The collected or simulated data is preprocessed and fed into a trained machine learning model (Random Forest or XGBoost) to predict the Air Quality Index (AQI) and provide health advisories accordingly.

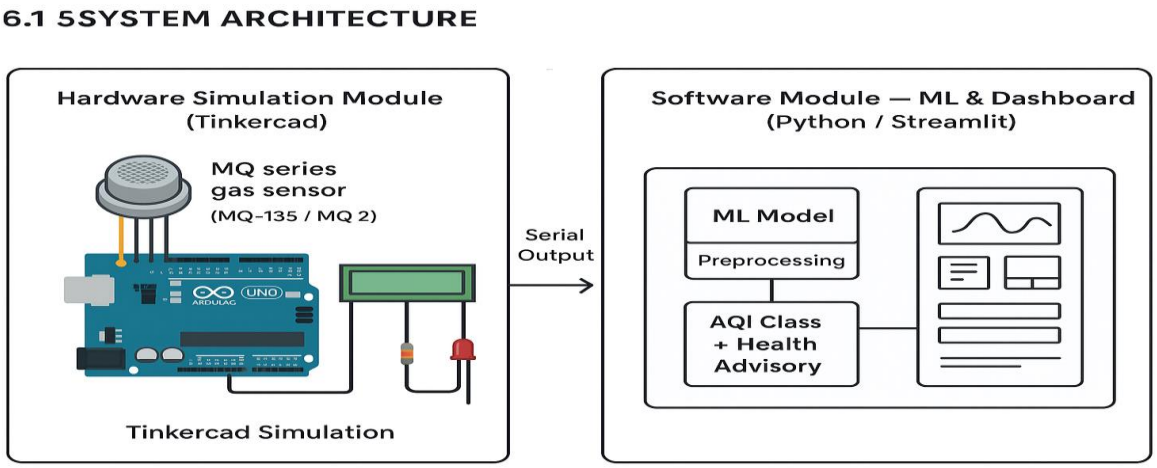


FIGURE 6.1 System Architecture

6.2 DATA FLOW DIAGRAM(DFD)

The Data Flow Diagram (DFD) illustrates the overall process of the **Air Quality Prediction System**. It shows how data moves to the final output display.

Sensor Input:

The air quality sensor (e.g., MQ-135) collects real-time gas concentration data and sends it to the Arduino for processing.

Data Preprocessing:

The raw sensor readings are converted into PPM (Parts Per Million) values and filtered to remove noise, ensuring accurate results.

Machine Learning / Logical Analysis:

Based on the sensor value range, the system classifies the air quality level as *Fresh*, *Poor*, or *Very Poor*.

Output Display:

The evaluated air quality level is displayed on the **LCD screen** and can also be visualized on a **dashboard** for better monitoring and analysis.

Alert System:

If the air quality crosses a defined threshold, an **LED or buzzer** is activated to warn users of poor air conditions.

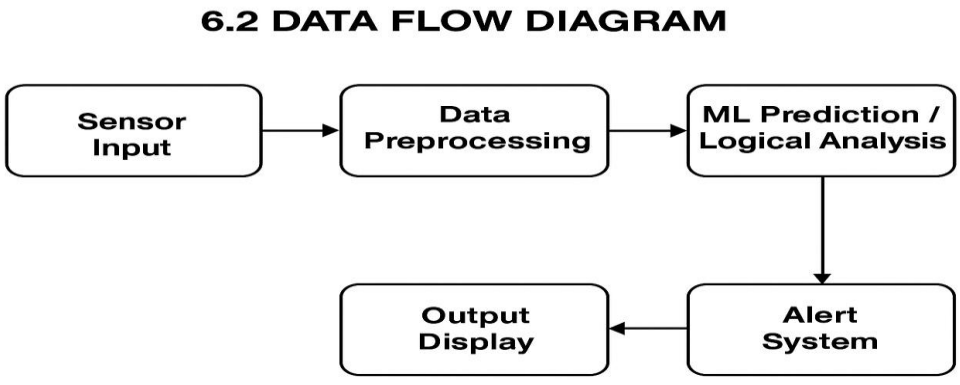


FIGURE 6.2 Data Flow Diagram

CHAPTER 7

SYSTEM IMPLEMENTATION

7.1 MODULES

Data Collection

Feature Engineering

Model Training

Prediction & Advisory

7.2 MODULE DESCRIPTION

7.2.1 Data Collection and Preprocessing

This module collects real-time air quality data from the MQ-135 sensor. The sensor measures gas concentration in the air and sends analog values to the Arduino.

7.2.2 Feature Engineering and Selection

Important parameters such as gas concentration, temperature, and humidity are identified. Only the most relevant features are selected to improve prediction accuracy and reduce unnecessary processing.

7.2.3 Machine Learning Model Development

A classification model is developed to predict air quality levels such as Good, Moderate, or Poor. The data is divided into training and testing sets, and the model learns patterns to classify new readings accurately.

7.2.4 Model Evaluation and Validation

The model is evaluated using accuracy, precision, and confusion matrix. These metrics ensure that predictions are consistent and reliable for real-time monitoring.

7.2.5 Algorithm

1. Read sensor value.
2. Convert to PPM.
3. Compare with threshold.
4. Display air quality message.
5. Activate alert if air is “Very Poor.”

CHAPTER 8

SYSTEM TESTING AND MAINTENANCE

8.1 UNIT TESTING

Unit testing is carried out on individual components of the Air Quality Prediction System to verify that each module functions correctly. The preprocessing steps, including decimal value replacement, conversion of -200 to NaN, interpolation of missing values, and extraction of time-related features, are tested separately.

8.2 INTEGRATION TESTING

Integration testing focuses on checking how individual modules interact with each other. The complete data flow is tested — from reading sensor data or CSV input to preprocessing, scaling, prediction by the model, and final display on the dashboard or LCD.

8.3 FUNCTIONAL TESTING

Functional testing validates that the system meets its intended functional requirements. The user interface elements such as sliders, prediction buttons, and file upload features are tested manually to confirm proper behavior. The model's predictions, displayed air quality levels, and health advisories are verified for accuracy.

8.4 SYSTEM TESTING

System testing evaluates the entire system as a single unit. The full workflow — from training the model to saving the trained artifacts, loading them into the dashboard, and predicting known test cases — is executed.

8.5 WHITE BOX TESTING

In white box testing, the internal logic and structure of the model are examined. Parameters such as the depth of the decision tree, number of features used, and the correctness of data scaling operations are verified

8.6 BLACK BOX TESTING

Black box testing treats the entire system as a closed entity without inspecting internal code. The accuracy of air quality labels and health advisories is verified against expected outcomes. This ensures the system's reliability and usability from an end-user perspective.

CHAPTER 9

FUTURE ENHANCEMENT AND CONCLUSION

Calibration and Real Sensors:

The system can be upgraded from basic MQ-type sensors to calibrated electrochemical or optical sensors. These provide more precise measurements and enable long-term deployment in outdoor environments.

Geo-Mapping and Visualization:

By integrating GPS-based data collection, sensor readings can be plotted on a map to visualize air quality across different areas. This feature can be extended to display live pollution heatmaps for smart city dashboards.

Edge Deployment:

The trained machine learning model can be converted to lightweight formats like ONNX or TensorFlow Lite and deployed on low-power devices such as Raspberry Pi or ESP32. This enables real-time edge inference without continuous cloud dependency.

CONCLUSION:

The proposed system provides a practical and scalable approach for monitoring air quality using IoT and machine learning. It promotes environmental awareness and supports smart city initiatives by providing timely air quality information and health advisories. With future improvements such as advanced sensors, predictive analytics, the system can evolve into a robust, real-time environmental monitoring solution that contributes to sustainable urban development.

APPENDIX

A.1 SOURCE CODE

 # train_model.py

```
import pandas as pd, numpy as np, os, joblib, warnings, seaborn as sns, matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split, RandomizedSearchCV
from sklearn.ensemble import RandomForestClassifier
from xgboost import XGBClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, f1_score, confusion_matrix

warnings.filterwarnings("ignore")
RND = 42
INPUT_FILE = "AirQuality.csv"
SAVE_MODEL = "final_model.joblib"
SAVE_SCALER = "scaler.joblib"

def co_to_label(v):
    if v < 1.0: return 0
    elif v < 2.5: return 1
    elif v < 5.0: return 2
    elif v < 8.0: return 3
    else: return 4

def load_data(p):
    if not os.path.exists(p): raise FileNotFoundError(f"Missing: {p}")
    return pd.read_csv(p, sep=';', decimal=',', header=0)

def preprocess(df):
    df = df.copy()
    df.columns = [c.strip() for c in df.columns]
    df.replace(-200, np.nan, inplace=True)
    df['Time'] = df['Time'].astype(str).str.replace('.', ':', regex=False)
    df['datetime'] = pd.to_datetime(df['Date'] + ' ' + df['Time'], dayfirst=True, errors='coerce')
    cols = ['CO(GT)','C6H6(GT)','NOx(GT)','NO2(GT)','T','RH','PT08.S1(CO)',
            'PT08.S2(NMHC)','PT08.S3(NOx)','PT08.S4(NO2)','PT08.S5(O3)','AH']
    cols = [c for c in cols if c in df.columns]
```

```
    for c in cols: df[c] = pd.to_numeric(df[c], errors='coerce')
    df[cols] = df[cols].interpolate(limit=5, limit_direction='both')
    df.dropna(subset=['CO(GT)'], inplace=True)
    df['hour'] = df['datetime'].dt.hour.fillna(0).astype(int)
    df['label'] = df['CO(GT)'].apply(co_to_label)
    return df

def train(df):
    feats = [c for c in ['T','RH','C6H6(GT)','NOx(GT)','NO2(GT)','hour',
                         'PT08.S1(CO)','PT08.S2(NMHC)','PT08.S3(NOx)',
                         'PT08.S4(NO2)','PT08.S5(O3)','AH'] if c in df.columns]
    X, y = df[feats].fillna(0), df['label']
    joblib.dump(feats, "features.joblib")
    Xtr, Xte, ytr, yte = train_test_split(X, y, stratify=y, test_size=0.2, random_state=RND)
    sc = StandardScaler(); Xtr_s = sc.fit_transform(Xtr); Xte_s = sc.transform(Xte)

    rf = RandomForestClassifier(random_state=RND)
    rf_params = {'n_estimators':[100,300,500],'max_depth':[10,20,None],
                 'min_samples_split':[2,5],'min_samples_leaf':[1,2],
                 'max_features':['sqrt','log2']}
    rf_cv = RandomizedSearchCV(rf, rf_params, n_iter=10, scoring='f1_weighted', cv=3, random_state=RND,
n_jobs=-1)
    rf_cv.fit(Xtr_s, ytr)
    rf_pred = rf_cv.best_estimator_.predict(Xte_s)
    rf_acc = accuracy_score(yte, rf_pred)
    rf_f1 = f1_score(yte, rf_pred, average='weighted')

    xgb = XGBClassifier(use_label_encoder=False, eval_metric='mlogloss', random_state=RND)
    xgb_params = {'n_estimators':[100,300],'max_depth':[3,6,10],
                  'learning_rate':[0.01,0.1,0.2],'subsample':[0.8,1.0],
                  'colsample_bytree':[0.8,1.0],'gamma':[0,0.2],
                  'min_child_weight':[1,3]}
    xgb_cv = RandomizedSearchCV(xgb, xgb_params, n_iter=15, scoring='f1_weighted', cv=3,
random_state=RND, n_jobs=-1)
    xgb_cv.fit(Xtr_s, ytr)
    xgb_pred = xgb_cv.best_estimator_.predict(Xte_s)
    xgb_acc = accuracy_score(yte, xgb_pred)
    xgb_f1 = f1_score(yte, xgb_pred, average='weighted')
```

```
    res = {'RandomForest': rf_f1, 'XGBoost': xgb_f1}
    best_name = max(res, key=res.get)
    best_model = rf_cv.best_estimator_ if best_name == 'RandomForest' else xgb_cv.best_estimator_
    best_acc = rf_acc if best_name == 'RandomForest' else xgb_acc
    best_f1 = res[best_name]

    joblib.dump(best_model, SAVE_MODEL)
    joblib.dump(sc, SAVE_SCALER)

    cm = confusion_matrix(yte, best_model.predict(Xte_s))
    plt.figure(figsize=(5,4))
    sns.heatmap(cm, annot=True, fmt='d', cmap='Greens')
    plt.xlabel("Predicted"); plt.ylabel("True")
    plt.title(f"Confusion Matrix ({best_name})")
    plt.savefig("confusion_matrix.png", dpi=150)

    print(f"Training Done\nBest Model: {best_name}\nAccuracy: {best_acc:.3f}\nF1 Score: {best_f1:.3f}")

if __name__ == "__main__":
    print("Loading:", INPUT_FILE)
    df_raw = load_data(INPUT_FILE)
    print("Raw shape:", df_raw.shape)
    df = preprocess(df_raw)
    print("Processed:", df.shape)
    train(df)
```

#app.py

```
import pandas as pd, numpy as np, os, joblib, warnings, matplotlib.pyplot as plt, seaborn as sns
from sklearn.model_selection import train_test_split, RandomizedSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier
from xgboost import XGBClassifier
from sklearn.metrics import accuracy_score, f1_score, confusion_matrix
warnings.filterwarnings("ignore")

RND = 42
```

```
INPUT_FILE = "AirQuality.csv"
SAVE_MODEL = "final_model.joblib"
SAVE_SCALER = "scaler.joblib"

def co_to_label(x):
    if x < 1.0: return 0
    elif x < 2.5: return 1
    elif x < 5.0: return 2
    elif x < 8.0: return 3
    else: return 4

def load_data(path):
    if not os.path.exists(path): raise FileNotFoundError("Input file not found")
    return pd.read_csv(path, sep=';', decimal=',', header=0)
def preprocess(df):
    df = df.copy()
    df.replace(-200, np.nan, inplace=True)
    df.columns = [c.strip() for c in df.columns]
    df['Time'] = df['Time'].astype(str).str.replace('.', ':', regex=False)
    df['datetime'] = pd.to_datetime(df['Date'] + ' ' + df['Time'], dayfirst=True, errors='coerce')
    cols = ['CO(GT)', 'C6H6(GT)', 'NOx(GT)', 'NO2(GT)', 'T', 'RH', 'PT08.S1(CO)', 'PT08.S2(NMHC)',
        'PT08.S3(NOx)', 'PT08.S4(NO2)', 'PT08.S5(O3)', 'AH']
    cols = [c for c in cols if c in df.columns]
    for c in cols: df[c] = pd.to_numeric(df[c], errors='coerce')
    df[cols] = df[cols].interpolate(limit=5, limit_direction='both')
    df.dropna(subset=['CO(GT)'], inplace=True)
    df['hour'] = df['datetime'].dt.hour.fillna(0).astype(int)
    df['label'] = df['CO(GT)'].apply(co_to_label)
    return df

def train(df):
    features = [c for c in ['T','RH','C6H6(GT)','NOx(GT)','NO2(GT)','hour','PT08.S1(CO)',
            'PT08.S2(NMHC)','PT08.S3(NOx)','PT08.S4(NO2)','PT08.S5(O3)','AH'] if c in df.columns]
    X, y = df[features].fillna(0), df['label']
    joblib.dump(features, "features.joblib")
    X_train, X_test, y_train, y_test = train_test_split(X, y, stratify=y, test_size=0.2, random_state=RND)
    scaler = StandardScaler()
    X_train, X_test = scaler.fit_transform(X_train), scaler.transform(X_test)
```

```
    rf = RandomForestClassifier(random_state=RND)
    rf_params = {'n_estimators':[100,300],'max_depth':[10,20,None],'min_samples_split':[2,5],
                 'min_samples_leaf':[1,2],'max_features':['sqrt','log2']}
    rf_cv = RandomizedSearchCV(rf, rf_params, n_iter=10, scoring='f1_weighted', cv=3, random_state=RND,
n_jobs=-1)
    rf_cv.fit(X_train, y_train)
    rf_pred = rf_cv.best_estimator_.predict(X_test)
    rf_acc, rf_f1 = accuracy_score(y_test, rf_pred), f1_score(y_test, rf_pred, average='weighted')
    xgb = XGBClassifier(use_label_encoder=False, eval_metric='mlogloss', random_state=RND)
    xgb_params = {'n_estimators':[100,300],'max_depth':[6,10],'learning_rate':[0.05,0.1],
                  'subsample':[0.9,1.0],'colsample_bytree':[0.8,1.0]}
    xgb_cv = RandomizedSearchCV(xgb, xgb_params, n_iter=10, scoring='f1_weighted', cv=3,
random_state=RND, n_jobs=-1)
    xgb_cv.fit(X_train, y_train)
    xgb_pred = xgb_cv.best_estimator_.predict(X_test)
    xgb_acc, xgb_f1 = accuracy_score(y_test, xgb_pred), f1_score(y_test, xgb_pred, average='weighted')

    best_model = rf_cv.best_estimator_ if rf_f1 > xgb_f1 else xgb_cv.best_estimator_
    joblib.dump(best_model, SAVE_MODEL)
    joblib.dump(scaler, SAVE_SCALER)

    cm = confusion_matrix(y_test, best_model.predict(X_test))
    plt.figure(figsize=(5,4))
    sns.heatmap(cm, annot=True, fmt='d', cmap='Greens')
    plt.xlabel("Predicted"); plt.ylabel("True")
    plt.title("Confusion Matrix")
    plt.savefig("confusion_matrix.png", dpi=150)

    print("Training done.")
    print(f"RandomForest: Acc={rf_acc:.3f}, F1={rf_f1:.3f}")
    print(f"XGBoost: Acc={xgb_acc:.3f}, F1={xgb_f1:.3f}")
  if __name__ == "__main__":
    print("Loading dataset:", INPUT_FILE)
    df = preprocess(load_data(INPUT_FILE))
    print("Processed shape:", df.shape)
    train(df)
```

```
#arduino

#include <LiquidCrystal.h>
LiquidCrystal lcd(12, 11, 5, 4, 3, 2);
int pin8 = 8;
 int analogPin = A0;
 int sensorValue = 0;
void setup() {
  pinMode(analogPin, INPUT);
  pinMode(pin8, OUTPUT);
  lcd.begin(16, 2);
  lcd.print("What is the air ");
  lcd.print("quality today?");
  Serial.begin(9600);
  lcd.display();
}
 void loop() {
  delay(1000);
  sensorValue = analogRead(analogPin);
  Serial.print("Air Quality in PPM = ");
  Serial.println(sensorValue);
  lcd.clear();
  lcd.setCursor(0,0);
  lcd.print ("Air Quality: ");
  lcd.print (sensorValue);
  if (sensorValue<=500)
   {
   Serial.print("Fresh Air ");
   Serial.print ("\r\n");
   lcd.setCursor(0,1);
   lcd.print("Fresh Air");
   }
  else if( sensorValue>=500 && sensorValue<=650 )
   {
   Serial.print("Poor Air");
   Serial.print ("\r\n");
   lcd.setCursor(0,1);
   lcd.print("Poor Air");
```

```
  }
 else if (sensorValue>=650 )
  {
  Serial.print("Very Poor Air");
  Serial.print ("\r\n");
  lcd.setCursor(0,1);
  lcd.print("Very Poor Air");
  }
 if (sensorValue >650) {
   digitalWrite(pin8, HIGH);
 }
 else {
   digitalWrite(pin8, LOW);
 }
}
```

A.2 SCREENSHOTS

```
PS D:\Machine Learning Project> python train_model.py
Loading dataset: AirQuality.csv
Raw shape: (9471, 17)
Processed shape: (8134, 20)

✅ Training Completed Successfully!
Random Forest - Accuracy: 0.864, F1 Score: 0.863
XGBoost - Accuracy: 0.880, F1 Score: 0.879
Best Model: XGBoost
Accuracy: 0.880
Weighted F1 Score: 0.879
Model saved as: final_model.joblib
Scaler saved as: scaler.joblib
Confusion matrix saved as: confusion_matrix.png
```

Figure A.2.1 Console output

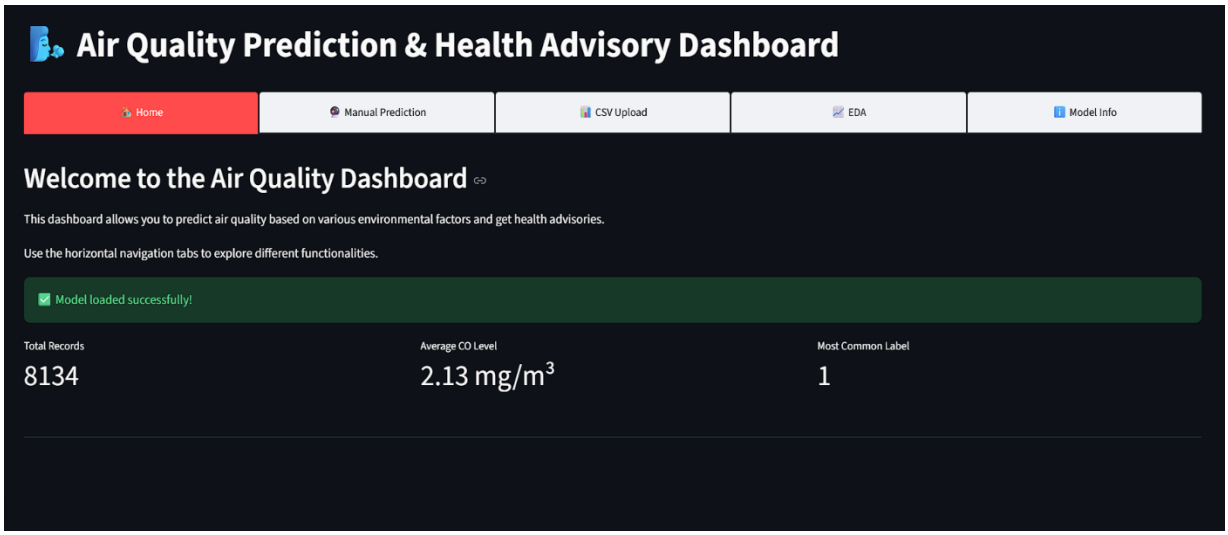


Figure A.2.2 Home Page

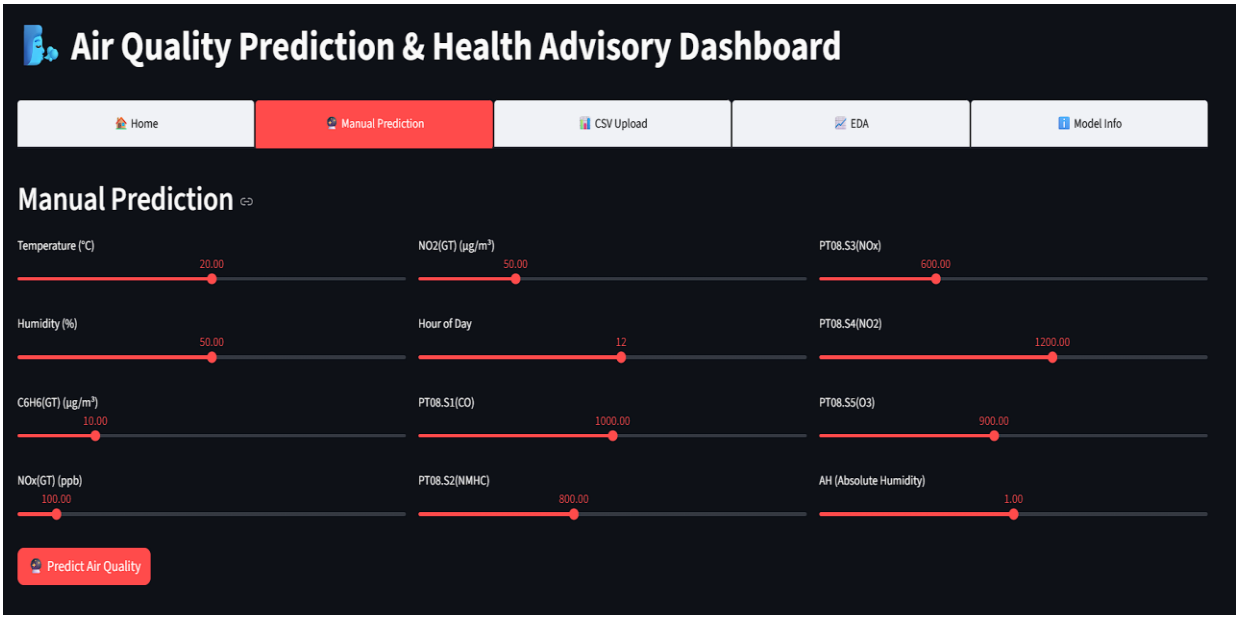


Figure A.2.3 Manual Prediction

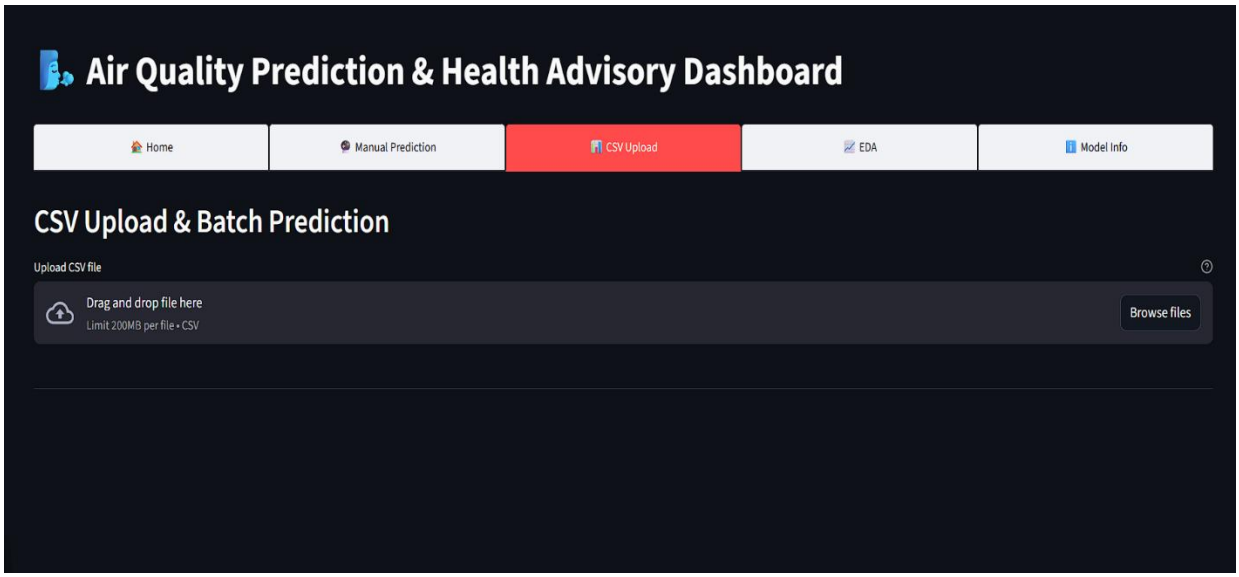
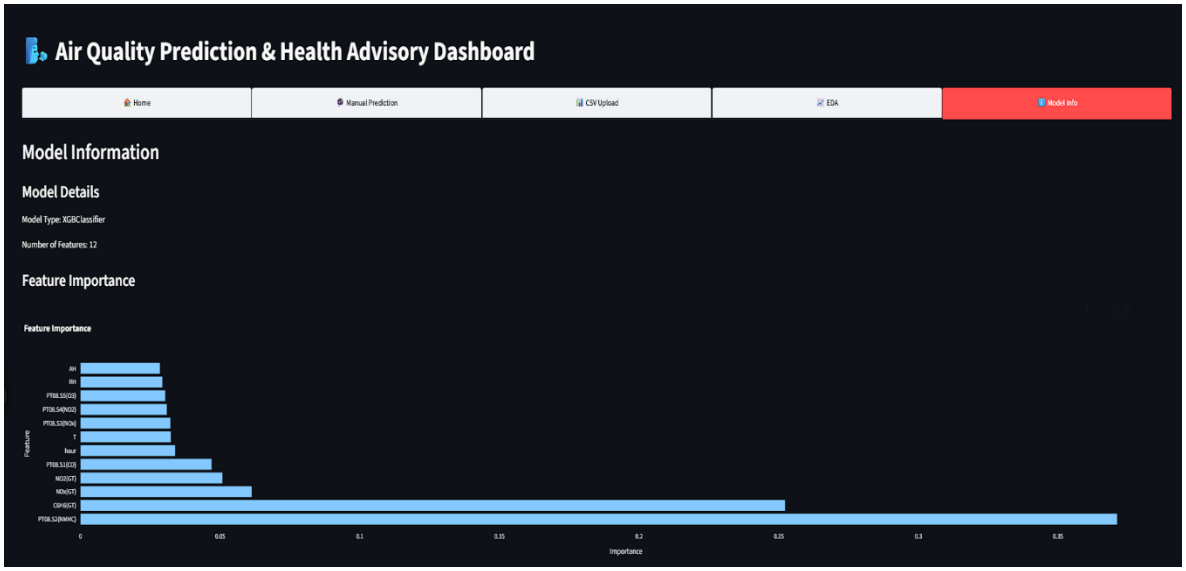


Figure A.2.4 EDA



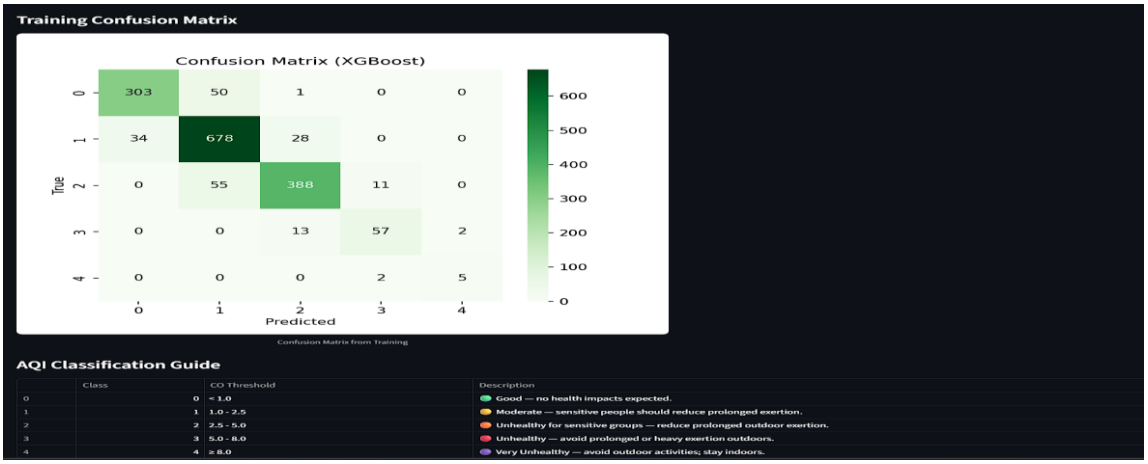
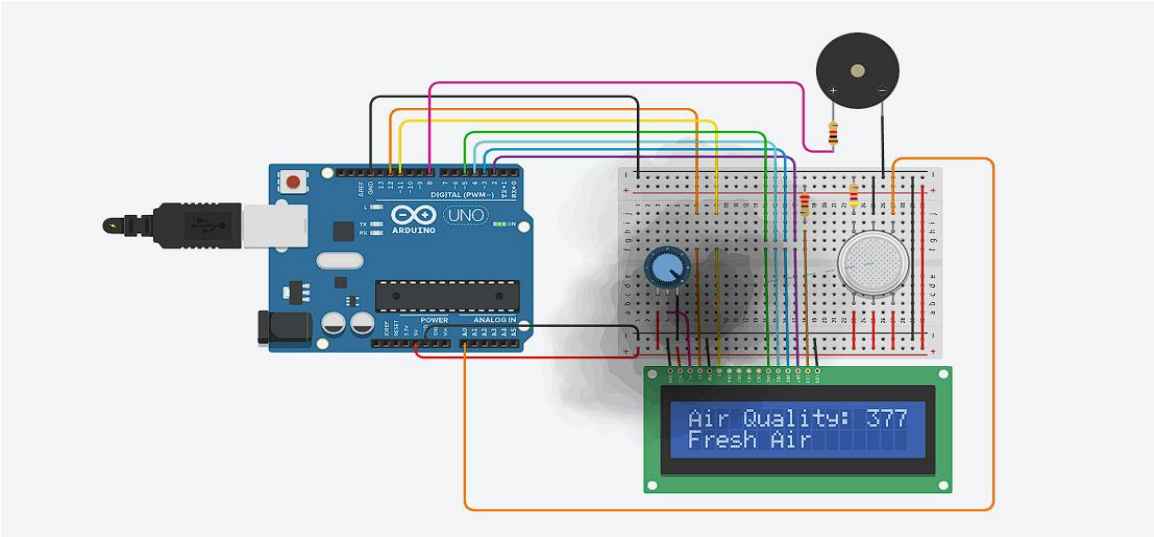


Figure A.2.5 Model Info



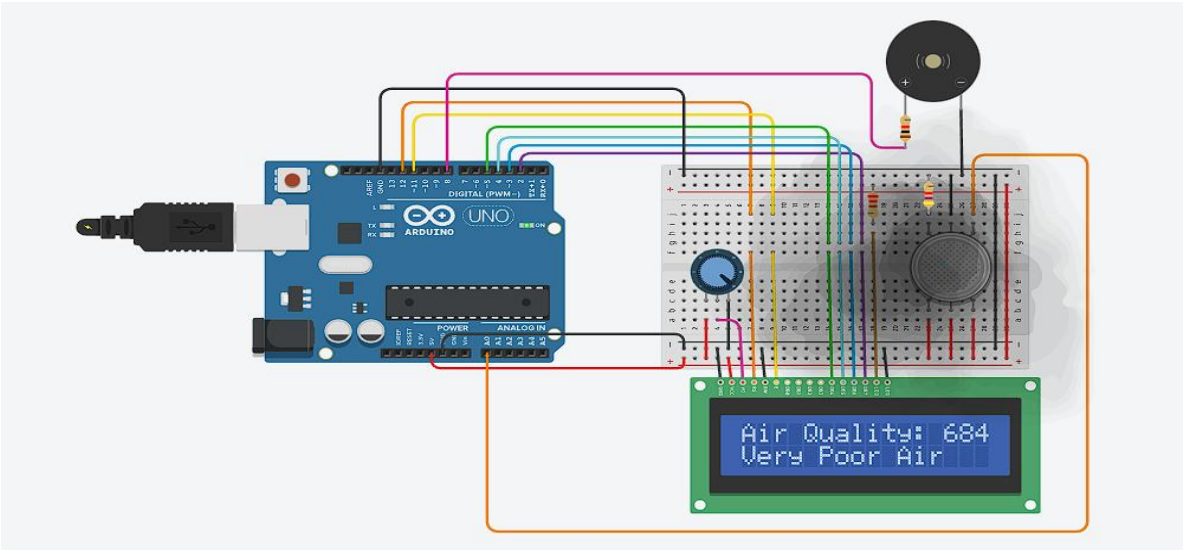


Figure A.2.6 Tinkercad

REFERENCES

1. Gupta, S., & Kumar, R. (2021). *Air Quality Prediction Using Machine Learning Techniques: A Review.* International Journal of Environmental Science and Technology, 18(3), 1023–1035.

2. Jain, A., & Sharma, N. (2022). *Implementation of Random Forest and XGBoost Models for Air Pollution Forecasting.* IEEE International Conference on Computational Intelligence and Data Science (ICCIDS).

3. World Health Organization (WHO). (2023). *Air Pollution and Health.* Retrieved from https://www.who.int/health-topics/air-pollution

4. U.S. Environmental Protection Agency (EPA). (2024). *Air Quality Index (AQI) Basics.* Retrieved from https://www.airnow.gov

5. Scikit-learn Developers. (2024). *Scikit-learn Documentation – Machine Learning in Python.* Retrieved from https://scikit-learn.org/stable/

6. XGBoost Developers. (2024). *XGBoost Documentation.* Retrieved from https://xgboost.readthedocs.io

7. Tinkercad. (2024). *Arduino and Sensor Simulation Platform.* Retrieved from https://www.tinkercad.com

8. Streamlit. (2024). *Streamlit: The Fastest Way to Build Data Apps in Python.* Retrieved from https://docs.streamlit.io

9. Pandas Development Team. (2024). *Pandas User Guide.* Retrieved from https://pandas.pydata.org

10. OpenAI. (2025). *AI-based Insights for Environmental Monitoring Systems.* Technical Resources, https://openai.com/research